

Rapport d'APS

shiyao chen 28707756

Avril 2024



Contents

1	Introduction	3
1.1	Objectifs du rapport	3
2	Installation et Utilisation du Projet	3
2.1	Protocole d'installation	3
2.2	Description de la structure	4
2.3	Instructions d'utilisation	4
3	Portée Globale du Projet	5
4	Choix d'Implémentations et Difficultés Rencontrées	7
4.1	Choix d'Implémentations	7
4.2	Difficultés Rencontrées	7
5	Conclusion	8

1 Introduction

Dans le domaine de l'informatique, la conception et la mise en œuvre de langages de programmation ont toujours été une tâche importante mais difficile. La conception des langages de programmation n'est pas seulement liée au développement de la théorie informatique, mais affecte également directement l'efficacité et la qualité du développement de logiciels. Dans ce contexte, nous avons commencé la conception et la mise en œuvre d'un langage de programmation appelé APS (Analyse des Programmes et Sémantique).

1.1 Objectifs du rapport

Le but de ce rapport est de présenter les résultats et les expériences réalisées par notre équipe dans la conception et la mise en œuvre du langage APS. Le langage APS est un projet évolutif que nous continuons d'étendre et d'améliorer pour devenir un outil de programmation puissant et facile à utiliser.

Dans ce rapport, nous donnerons d'abord un aperçu des différentes versions du langage APS, notamment APS0, APS1, APS1a et APS2. Nous discutons ensuite en détail des détails de conception et de mise en œuvre de chaque version, y compris la syntaxe, la sémantique, les fonctionnalités clés et les solutions. Enfin, nous résumerons l'ensemble du projet et discuterons de l'orientation future du développement du langage APS.

Grâce à ce rapport, nous espérons pouvoir démontrer de manière exhaustive la réflexion et les efforts de notre équipe dans le processus de conception et de mise en œuvre du langage APS, tout en fournissant une référence et une inspiration précieuses aux lecteurs intéressés.

2 Installation et Utilisation du Projet

2.1 Protocole d'installation

- Clonez le référentiel du projet APS depuis GitLab en utilisant la commande suivante :

```
git clone https://stl.algo-prog.info/28707756/projet_aps.git
```

- Accédez au répertoire du projet APS :

```
cd projet_APS
```

2.2 Description de la structure

Notre projet se trouve dans un répertoire nommé APS, qui est divisé en 4 sous-répertoires, APS0, APS1, APS1a et APS2, qui correspondent respectivement à l'implémentation du noyau de fonctions et du noyau de commandes. Chaque répertoire comprend :

- Niveau syntaxe : le lexique défini dans le fichier `lexer.mll` et la syntaxe définie dans les fichiers `parser.mly` et `ast.ml`, où le fichier `ast.ml` résume toutes les structures de notre langage.
- Niveau type : La méthode d'affichage de la syntaxe des fichiers `.aps` comme termes disponibles dans Prolog est définie dans le fichier `prologTerm.ml`, et l'analyse de type (qui analyse les résultats donnés par `prologTerm`) est définie dans le fichier `typepage.pl`.
- Niveau sémantique : Le fichier `eval.ml` définit la valeur ou l'effet attendu de l'exécution du programme de langage. De plus, nous pouvons trouver un Makefile qui compile `PrologTerm` ou l'évaluateur.

2.3 Instructions d'utilisation

Pour chaque sous-dossier de votre projet correspondant à un niveau d'implémentation du langage APS (0, 1, 1a, 2), suivez ces étapes pour tester le typage du langage APS et l'évaluer. Nous disposons d'un script `test.sh` dont la fonction est d'appliquer ce test sur tous les fichiers de test qui se trouvent dans le sous-dossier "Samplesi". Tous ces exemples sont syntaxiquement et sémantiquement corrects, ainsi que bien typés.

- Accédez au sous-répertoire du projet APS :

```
cd APSi\ML
```

- Créez les exécutable : :

```
make
```

- Lancez le typeur et l'évaluateur sur tous les fichiers de de samples :

```
chmod +x ./test.sh
./test.sh
```

- Pour supprimer les exécutable :

```
make clean
```

Il est également possible de tester un seul fichier .aps avec les instructions suivantes :

- Accédez au sous-répertoire du projet APS :

```
cd APSi\ML
```

- Créez les exécutables : :

```
make
```

- Génération de code en Prolog :

```
./prologTerm ../Samplesi/testi.aps
```

- Typage :

```
swipl typage.pl
?- main.
?- insérez le résultat de la génération de code en Prolog.
```

- Evalueur :

```
./eval ../Samplesi/testi.aps
```

3 Portée Globale du Projet

1. APS0 : Noyau Fonctionnel

- APS0 est principalement un langage d'expressions utilisé pour manipuler des entiers et des booléens.
- Il possède des caractéristiques fonctionnelles où les fonctions sont considérées comme des valeurs pouvant être passées en tant que paramètres ou résultats de calcul.
- APS0 permet de définir des constantes ou des fonctions et de les lier à des identificateurs.

2. APS1 : Noyau Impératif

- APS1 étend les fonctionnalités d'APS0 en ajoutant des éléments du langage impératif tels que les instructions d'assignation, les structures conditionnelles et les boucles.
- Il offre davantage de moyens d'exprimer les résultats de calcul, mais ne diffère

pas beaucoup d'APS0 en termes de capacités de calcul.

- APS1 introduit les instructions d'assignation, levant ainsi les limitations des constantes.

Par exemple :

```
[
  VAR x int;
  SET x 0;
  PROC p [y:int]
  [
    WHILE (lt x y) [
      ECHO x;
      SET x (add x 1)
    ]
  ];
  CALL p 2
]
```

3. APS1a : Valeur ou Référence

- APS1a étend APS1 pour résoudre certains problèmes de contraintes de type.
- Il introduit la distinction entre constantes et variables, ainsi que le concept de types de référence, renforçant ainsi l'expressivité et la sécurité du système de types.

Par exemple :

```
[
  PROC inc [var x:int] [ SET x (add x 1) ];
  VAR n int;
  SET n 41;
  CALL inc (adr n);
  ECHO n
]
```

4. APS2 : Tableaux

- APS2 ajoute la prise en charge des tableaux, qui sont des séquences de valeurs stockées dans des adresses mémoire consécutives.
- Il fournit des primitives d'allocation mémoire explicites, faisant des tableaux une nouvelle valeur de type dans le langage, pouvant être trouvée dans l'environnement ou la mémoire.
- Les tableaux sont des structures modifiables, ce qui affecte les instructions d'assignation en permettant aux éléments de gauche d'être des identificateurs ou des éléments de tableau.

Par exemple :

```
[
```

```

CONST xs (vec int) (alloc 5);
VAR i int;
SET i 0;
WHILE (lt i (len xs))
[
  SET (nth xs i) (add i 1);
  SET i (add i 1)
];
ECHO (nth xs 0);
ECHO (nth xs 1);
ECHO (nth xs 2);
ECHO (nth xs 3);
ECHO (nth xs 4)
]

```

4 Choix d'Implémentations et Difficultés Rencontrées

4.1 Choix d'Implémentations

Nous avons opté pour OCaml pour le développement d'APS, en raison de sa nature à typage statique qui permet de détecter de nombreuses erreurs de programmation lors de la compilation, améliorant ainsi la fiabilité et la robustesse du code. De plus, OCaml est un langage de programmation fonctionnel qui offre des fonctionnalités telles que les fonctions d'ordre supérieur, les fermetures et la récursivité, facilitant ainsi la création de programmes hautement abstraits.

Pour le typage, nous avons choisi d'utiliser Prolog, car Prolog offre un puissant mécanisme de correspondance de motifs qui permet de faire correspondre plus facilement des modèles et des types, ce qui est très utile pour le développement du système de typage. De plus, la syntaxe de Prolog est très flexible, ce qui nous permet de nous étendre facilement à des systèmes de types plus complexes. De plus, le code Prolog est généralement plus concis que celui des autres langages de programmation, car ses règles et ses requêtes peuvent être exprimées de manière concise.

4.2 Difficultés Rencontrées

Nous avons réussi à implémenter entièrement APS0, APS1, APS1a et APS2, y compris le typage et l'évaluation pour tous les niveaux d'APS. De plus, tout fonctionne correctement dans APS0, APS1 et APS1a.

Cependant, dans APS2, tout fonctionne sauf la gestion des tableaux à plusieurs dimensions, tels que `(vec (vec bool))`, au niveau de l'évaluateur. Nous devons corriger la grammaire pour prendre en charge cette fonctionnalité.

5 Conclusion

APS est un langage de programmation polyvalent, offrant à la fois la simplicité nécessaire pour faciliter l'apprentissage des débutants en informatique et la puissance requise pour permettre la manipulation avancée de données et la résolution de problèmes complexes. Le langage APS est conçu pour être utilisé dans un cadre pédagogique, mais il est également adaptable à des applications pratiques dans le domaine de l'informatique.

Dans ce rapport, nous avons examiné en détail le projet APS, en mettant en évidence ses différentes composantes et en discutant des choix de conception, des difficultés rencontrées et des solutions apportées.

En conclusion, le projet APS a été une expérience enrichissante qui nous a permis de mettre en pratique nos connaissances en programmation fonctionnelle et impérative. Malgré les défis rencontrés, nous avons réussi à réaliser l'intégralité du projet et à surmonter les obstacles grâce à une collaboration étroite et à un travail d'équipe efficace.

Pour l'avenir du projet, nous envisageons d'ajouter des fonctionnalités supplémentaires telles que la gestion avancée des erreurs, la prise en charge de nouveaux types de données et l'optimisation des performances. De plus, nous chercherons à améliorer la documentation du projet et à étendre la base de tests pour assurer la qualité du code APS. Enfin, nous resterons ouverts aux retours des utilisateurs et aux contributions de la communauté pour faire évoluer le projet APS dans la bonne direction.